SCHEDULING WITH OUT RTOS

OVERVIEW

This project implements a simple cooperative multitasking scheduler on an Arduino Uno using Timer0 interrupts. Instead of relying on delay(), which blocks the CPU, the program sets up Timer0 in CTC (Clear Timer on Compare) mode to generate an interrupt every 1 millisecond. The prescaler is set to 64 to slow down the CPU clock for the timer, and the compare register (OCR0A) is set to 249 to achieve exactly a 1 ms interrupt period. This makes the timer act like a “heartbeat” that drives all the scheduled tasks.

Inside the interrupt service routine (ISR), a global tick counter (systemTick) increments every millisecond. Based on the value of this counter, flags are set for different tasks: the sensor task every 100 ms, the display update every 500 ms, and the UART communication every 1000 ms. By resetting these flags in the main loop, the tasks are guaranteed to execute at their defined intervals without interfering with each other. The tick counter is reset every 1000 ms to keep it from overflowing and to align the schedule.

The main program loop continuously checks these task flags. When a flag is set, the corresponding task runs—printing a message to the Serial Monitor in this example—and then the flag is cleared. This ensures that the tasks execute only when they are scheduled, while the loop remains non-blocking. The cooperative design means the tasks must be short and efficient, but it keeps the system responsive, unlike a blocking delay-based approach.

Overall, this design demonstrates how to implement a lightweight scheduler on a microcontroller without a real-time operating system (RTOS). It highlights the power of hardware timers and interrupts for precise timing and task management. Such an approach is useful for small embedded systems where multiple periodic tasks, such as sensor sampling, display refreshing, and communication, need to run concurrently and predictably. It lays the foundation for building more advanced real-time systems.

how i did it

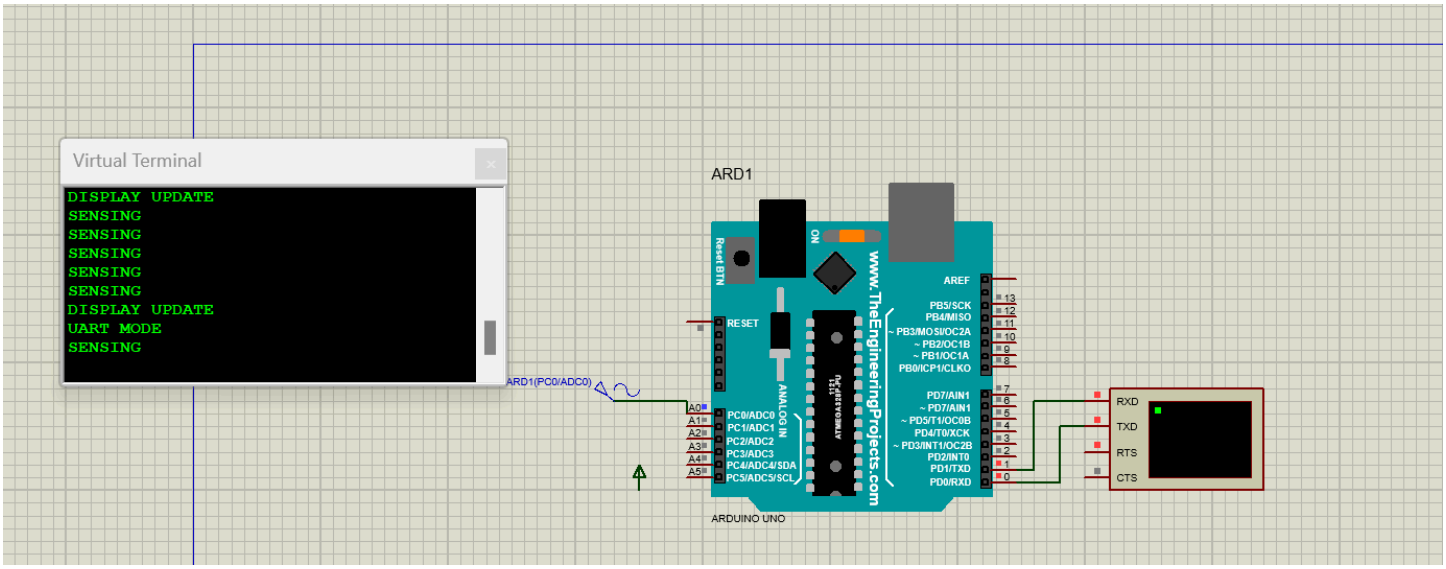
I implemented a interupt based timer meaning I set isr to keep updating a variable ever millisecond to act a pseudo timer here there is no task priority so if a task take too long the task may get missed thats bug other than that it works fine

source code:

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile unsigned long systemTick = 0;
volatile bool sensorFlag = false;
volatile bool displayFlag = false;
volatile bool uartFlag = false;

ISR(TIMER0_COMPA_vect) {
  systemTick++;

  if (systemTick % 100 == 0) sensorFlag = true;
  if (systemTick % 500 == 0) displayFlag = true;
  if (systemTick % 1000 == 0) uartFlag = true;

  if (systemTick >= 1000) systemTick = 0;
}

void timerInit() {
  TCCR0A = (1 << WGM01);
  OCR0A = 249;
  TCCR0B = (1 << CS01) | (1 << CS00);
  TIMSK0 = (1 << OCIE0A);
  sei();
}
```

```
void setup() {
   Serial.begin(9600);
   timerInit();

}

void loop() {
if (sensorFlag) {
      Serial.println("SENSING");
      sensorFlag = false;
   }

   if (displayFlag) {
      Serial.println("DISPLAY UPDATE");
      displayFlag = false;
   }

   if (uartFlag) {
      Serial.println("UART MODE");
      uartFlag = false;
   }

}
```

output


Virtual Terminal
DISPLAY UPDATE
SENSING
SENSING
SENSING
SENSING
SENSING
DISPLAY UPDATE
UART MODE
SENSING
ARD1
ARD1(PC0/ADC0)
Reset BTN
ON
RESET
ANALOG IN
AREF
PB5/SCK
PB4/MISO
~ PB3/MOSI/OC2A
~ PB2/OC1B
~ PB1/OC1A
PB0/ICP1/CLKO
PD7/AIN1
~ PD7/AIN1
~ PD5/T1/OC0B
PD4/T0/XCK
~ PD3/INT1/OC2B
PD2/INT0
PD1/TXD
PD0/RXD
PC0/ADC0
PC1/ADC1
PC2/ADC2
PC3/ADC3
PC4/ADC4/SDA
PC5/ADC5/SCL
www.TheEngineeringProjects.com
ARDUINO UNO
RXD
TXD
RTS
CTS